

Pipeline All-in-One (ollie_wan)

Il seguente notebook ha lo scopo di spiegare la pipeline seguita per la preparazione all'addestramento della rete neurale presente in `pix2pix.py`. Questo è un passaggio non fondamentale, di fatto inizialmente erano presenti diversi script, ancora reperibili nella sezione della repository **AGGIUNGI SEZIONE OLD_SCRIPTS/**, che successivamente sono stati implementati in questo unico file. La prima parte del nome `ollie_wan` è stata spiegata nel titolo: è un omofonia. La seconda `kenobi` segue la stessa linea di pensiero, portando il lettore a ripensare a un personaggio alquanto noto di una famosa saga.

1 Passaggi preliminari

1.1 Pacchetti necessari

I pacchetti necessari all'avvio del notebook sono:

- OpenCV → `cv2`: scaricabile con `pip install opencv-python`
- NumPy → `numpy`: scaricabile con `pip install numpy`

```
# Pacchetti standard
import os, random, argparse, json
from pathlib import Path
from typing import Optional

# pacchetti esterni
import cv2
import numpy as np
```

2 Localizzazione messaggi (ITA/EN)

Uno degli aspetti che abbiamo deciso di curare con particolare attenzione all'interno del codice `ollie_wan_kenobi.py` è la gestione della lingua, sia per i messaggi mostrati in fase di esecuzione, sia per i commenti lasciati nel codice, ai quali abbiamo dedicato altrettanta importanza.

Abbiamo separato tutti i messaggi rivolti all'utente in due file **JSON** contenuti all'interno della cartella `json/`:

- `messages.json`: contiene i messaggi interattivi che vengono mostrati durante l'esecuzione dello script;
- `help.json`: contiene i messaggi dedicati all'aiuto da linea comando mostrabile con `python ollie_wan_kenobi.py -h` oppure con `python ollie_wan_kenobi.py --help`;

Questo approccio ci ha permesso non solo di mantenere il codice più pulito e modulare, ma anche di aggiungere facilmente nuove lingue in futuro senza dover modificare direttamente il codice python.

L'implementazione di questa funzionalità è illustrata nel codice sottostante:

```
from pathlib import Path
import json
```

```
# Otteniamo il percorso della directory corrente
script_dir = Path(__file__).parent.resolve()

# Carichiamo il file di configurazione
messages_path = script_dir / "json" / "messages.json"
with messages_path.open("r", encoding="utf-8") as m:
    MESSAGES = json.load(m)
```

L'utente ha la possibilità di selezionare la lingua desiderata tramite il parametro `--lang` specificando successivamente `it` per la lingua italiana, oppure `en` per quella inglese

3 Parsing e struttura dei parametri

Per quanto riguarda l'esecuzione e più precisamente la chiamata del programma da riga comando, lo script sfrutta il modulo `argparse` per gestire i parametri in ingresso.

Sono stati definiti tre argomenti principali:

- `path`: che specifica il percorso della cartella contenente le immagini *label-free* e *stained* (si consiglia di inserire tale cartella in `Virtual-Staining/your_images_folder`). **È l'unico argomento obbligatorio**;
- `--seed SEED`: che specifica l'assenza o la presenza di un seed prestabilito. In caso affermativo si digiterà il numero del seed di seguito al flag (es: `--seed 42`). Questo approccio ci permette di garantire la riproducibilità dei risultati;
- `--save_mask`: una flag booleana opzionale che indica se salvare o meno le maschere binarie delle sottoimmagini calcolate. La presenza di questa flag indica il salvataggio delle maschere.

A questi si aggiunge il l'argomento `--lang` che come detto prima consente di scegliere la lingua desiderata.

L'utilizzo di `argparse` permette di generare automaticamente una guida all'uso del programma, visualizzabile con il flag `-h` o `--help`.

Dal punto di vista pratico, la sintassi per eseguire lo script è la seguente:

```
python ollie_wan_kenobi.py path [--seed SEED] [--save_masks] [--lang {en, ita}]
```

4 Estrazione delle sottoimmagini (patch)

Un passaggio fondamentale nella costruzione del dataset è l'estrazione delle sottoimmagini, o **patch**, a partire dalle immagini ad alta risoluzione. Dopo aver calcolato le maschere e allineato le immagini, il nostro obiettivo è quello di convertire queste immagini in coppie più piccole e gestibili, da utilizzare direttamente in fase di addestramento della rete neurale.

.

.

.

COME ABBIAMO FILTRATO? IMMAGINE TROPPO PICCOLA (QUAND'È TROPPO PICCOLA?) E TROPPO NERA (VA PRECISATO CHE IL NERO INDICA LO SFONDO, SI FACCIA RIFERIMENTO AL NOTEBOOK ALLINEAMENTO.IPYNB)

CURIOSITÀ MIA: PER CHE CAZZO STA `roi_mask`?

CHE DIFFERENZA C'È TRA `DIVIDE_WITH_GRID` E `DIVIDE_WITH_POSITION`?

METTIAMO CODICE BANALISSIMO (MA UTILE PER SPEZZARE L'EVENTUALE WALL OF TEXT) DI `EXTRACT_IAMGE`, COSÌ DICIAMO CHE TRATTIAMO LE IMMAGINI COME SEMPLICI MATRICI

5 Rinomina e salvataggio

6 Suddivisione in training/validation/test

7 Come possiamo aumentare il numero di immagini del nostro dataset? non solo diminuendo la dimensione delle patch